

Sleep-time compute란 무엇이며 왜 training인가

어려운 Study paper를 그림과 예로 다시 읽는 안내서

읽는 법 이 문서는 원 Study의 결론을 바꾸지 않고, 질문→세 문장→직관→예/반례→그림→대안→미증명 지점 순서로 풀어 쓴다. 파란색 training 용어는 별도 Background paper의 해당 개념으로 연결된다. 주장 ID와 figure ID는 원 Study와 동일하다.

Contents

E01 · Sleep-time compute 란 무엇이며 왜 training 인가	2
이 권이 답할질문	2
먼저 세 문장	2
직관	2
예와 반례	3
그림 읽기	4
대안과 비교	4
아직 모르는것	5
더 깊이 읽기	5
문서 경계와 사용법	7

E01 · Sleep-time compute 란 무엇이며 왜 training 인가

이 권이 답할질문

“모델이 잔다” 는 표현을 잠시 버리고도 sleep-time compute 를 정확히 정의할수 있을까? 답은 **사용자 응답이 끝난 뒤 계산한다**는 시간 조건과, 그 계산이 **다음 wake 에서 재사용될 상태를 바꾼다**는 상태 조건을함께 보는 것이다. 단순한 야간 batch job, 로그 압축, model checkpoint 복사는 첫 조건만 만족할 뿐이다. 반대로 응답 도중의 chain-of-thought 나 긴추론은 계산량이 커도 다음 episode 에 남지 않으면 두 번째 조건을 만족하지못한다.

이 권은 Study 의 operational definition 과 training map 을 쉬운 말로 연결한다. 핵심 근거는 **STC-C005 · synthesis · confidence high**, **STC-C006 · synthesis · confidence high**, **STC-C015 · synthesis · confidence high** 이다.

먼저 세 문장

1. Sleep-time compute 는 latency-critical response 뒤로 미룬계산이 미래의 reusable state 를 바꾸는 lifecycle 이다. “sleep” 이라는이름보다 이 두 조건이 중요하다.
2. 바뀌는 state 는 foundation weight 만이 아니다. adapter, recurrent state, text/vector memory, temporal graph, summary, index 도 가능하므로언제 갱신하는가와 어디에 기억하는가는 서로 독립된축이다.
3. state 를 경험에 맞춰 개선하려면데이터→순전파→손실→역전파→옵티마이저갱신 중 일부 또는 전부가 필요하다. 그래서 sleep 은 inference 의 부속기능이 아니라 작은 training system 에 가깝다.

한 줄 요약

Wake 가 “지금 답하기” 라면 sleep 은 “다음번에 더 잘 답하도록 재사용상태를 고치기” 다.

직관

두 개의 경계선을 그리면된다

첫 번째 경계는 **응답 SLA** 다. 사용자가 기다리는 동안 반드시 끝나야하는 계산이 wake path 이고, 사용자가 답을 받은 뒤 실행해도 되는 계산이 deferred path 다. 두 번째 경계는 **episode** 다. 계산 결과가 현재응답에서 사라지는지, 다음 대화 · 다음 task · 다음 날에도 재사용되는지를나눈다.

계산	응답 뒤 실행	다음 episode 에 재사용	STC 인가
응답 중 더 오래 생 각하기	아니오	대개 아니오	아니오
로그를 cold storage 로 복사	예	행동을 바꾸지 않음	보통 아니오
대화에서 stable preference 를 추 출해 memory store 에 기록	예	예	예, external-memory STC
replay 로 adapter 를 갱신 하고 새 version 을 배포	예	예	예, parametric STC

계산	응답 뒤 실행	다음 episode 에 재사용	STC 인가
search index 를 rebuild 해 이후 retrieval 을 개선	예	예	예, state/index STC

직관

“밤에 들었다”가 아니라 “다음번 행동을 바꾸는 상태가 남았다”가 판정기준이다. 이 정의를 쓰면 마케팅 용어가 달라도 dreaming, reflection, synthesis, consolidation 을 같은 좌표계에 놓을 수 있다.

Training 은 weight update 보다 넓다

Training 을 “거대한 pretraining cluster 에서 모든 weight 를 바꾸는 일”로만 보면 external-memory sleep 은 training 이 아닌 것처럼 보인다. 그러나 시스템 관점의 training 은 **경험으로부터 미래 행동을 개선할 state transformation 을 선택하는 일**이다. text summary 를 새로 만들 때도 어떤 episode 를 넣고 뺄지, 충돌을 어떻게 합칠지, 품질을 어떤 evaluator 로 판단할지에 objective 가 숨어 있다.

Parametric sleep 은 그 objective 를 gradient 로 직접 최적화한다. External sleep 은 LLM extraction, deduplication, clustering, graph update 처럼 더이산적인 procedure 를 쓴다. 둘 다 오류가 누적될 수 있고 validation·rollback·provenance 가 필요하다. 차이는 “학습인가 아닌가”보다 학습 결과가 어느 **medium** 에 저장되고 얼마나 되돌리기 쉬운가다.

예와 반례

예 1 — 외부 기억공고화

한 사용자가 여러 대화에서 “회의 요약은 결론부터, 표는 싫다”고 반복했다고 하자. sleep job 은 episode 들을 모아 중복을 제거하고, 일시적 요청과 stable preference 를 구분하고, “결론 우선 · 표 지양”이라는 durable memory 를 만든다. foundation weight 는 고정이지만 다음 대화에서 retrieval 되어 행동을 바꾸므로 STC 다. 장점은 원 episode 와 연결해 수정 · 삭제하기 쉽다는 점이다.

예 2 — adapter 로 선택적승격

한 공장 line 에서 반복되는 error code 와 복구 절차가 수천 번 재사용된다면, 매번 긴 문서를 retrieve 하는 비용보다 domain adapter 로 compile 하는 편이 유리할 수 있다. sleep job 은 검증된 record 만 골라 증류 또는 fine-tuning 으로 작은 delta 를 만들고, canary 평가를 통과한 뒤 활성화한다. 이것은 더 빠를 수 있지만 삭제 · 오염 · rollback 은 어려워진다.

반례 1 — 긴 test-time reasoning

한 질문에 100 배 많은 token 을 써서 답을 찾았지만 그 reasoning 이 버려졌다면 test-time compute 이지 STC 는 아니다. 같은 reasoning 에서 reusable strategy 를 추출해 다음 task 의 strategy memory 로 남겼을 때 비로소 lifecycle 의 sleep branch 가 된다.

Wake-test-sleep lifecycle

경계는 clock time이 아니라 response-critical dependency다

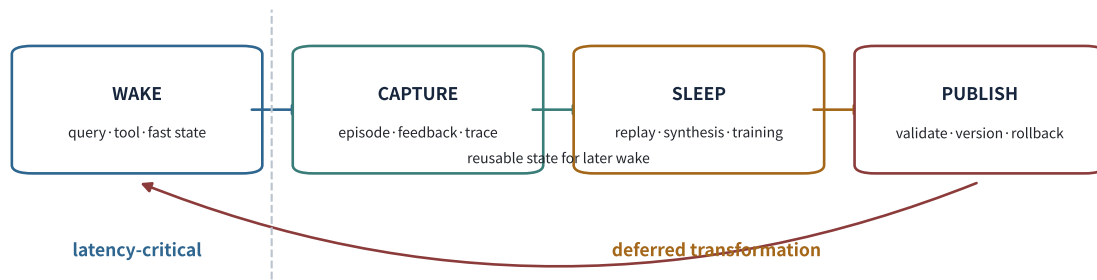


Figure 1: STC-F001. Wake-test-sleep lifecycle and reusable-state boundary. 원 Study 의 저자 작성 합성 figure 이며, 사실 주장의 인용은 원 Study 와 figure ledger 를 따른다.

반례 2 — 무조건적인 nightly fine-tuning

매일 밤 모든 user log 로 LoRA 를 학습한다고 해서 좋은 STC 가 되는 것은 아니다. 잘못된 label, transient intent, privacy-sensitive record 까지 섞이면 다음날 성능이 악화된다. “뒤에서 학습했다” 는 사실은 mechanism 이고, 무엇을 admit 하고 무엇을 보존하며 어떻게 되돌릴지가 solution 이다.

주의

Sleep-time compute 는 자동으로 안전하거나 효율적이라는 뜻이 아니다. latency budget 을 sleep budget 으로 옮겼을 뿐, 잘못된 학습의 위험과 비용은 사라지지 않는다.

그림 읽기

이 그림은 pretraining/post-training 이후의 배포 lifecycle 을 wake 와 sleep 으로 다시 자른다. 왼쪽에서 wake 는 요청 처리, retrieval, fast-state update 를 담당한다. 오른쪽 sleep 은 replay · reflection · distillation · compaction 을 실행한다. 가운데 reusable-state boundary 가 가장 중요하다. 그 경계를 넘는 object 가 text 인지 graph 인지 adapter 인지 에 따라 storage 와 governance 가 달라진다.

두 번째 그림은 “시간” 과 “medium” 을 직교축으로 놓는다. wake-time weight update 도 가능하고 sleep-time external-memory update 도 가능하다. 따라서 “parametric vs external” 과 “wake vs sleep” 을 한 축으로 섞으면 안 된다. 실제 system 은 네 사분면을 동시에 쓸 수 있다: wake 중 recurrent state, 응답 후 summary, 주기적 graph compaction, 드물게 adapter promotion.

대안과 비교

Pretraining·post-training·test-time·sleep-time 의 역할

phase	대표 데이터	바뀌는 것	강점	약점
pretraining	대규모 일반 corpus	foundation weights	넓은 prior	매우 비싸고 최신성 낮음

phase	대표 데이터	바뀌는 것	강점	약점
post-training	instruc- tion·prefer- ence·domain data	weights/adapter	행동 정렬	배포 전 분포에 묶임
wake/test-time learning	현재 con- text·feedback	activation/fast state/weight	즉시 적응	응답 latency 와 interference
sleep-time learning	누적 episode·re- play·derived data	external/para- metric reusable state	batch·검증·공 고화	staleness, pipeline 복잡도

Sleep 의 고유 장점은 **현재 응답의 latency 와 학습 계산을 분리**하고, 여러 episode 를 한꺼번에 비교해 dedup·conflict resolution·evaluation 할 수있다는 점이다. 반면 즉시 반영이 필요한 정보에는 늦다. 그래서 최선의 framing 은 네 phase 중 하나를 승자로 고르는 것이 아니라, 각 state 를 어느 phase 에서 갱신할지 결정하는 것이다.

Fixed 와 session-specific weights

Model artifact 를 foundation weights, shared domain adapter, user adapter, external memory, ephemeral state 로 분리하면 위험이 보인다. foundation 은 느리고 검증이 엄격해야 한다. user adapter 는 빠르게 personalized 될 수 있지만 tenant isolation 과 version explosion 이 생긴다. external memory 는 설명 가능하지만 retrieval latency 와 context budget 을 쓴다. “모든 기억을 weight 로” 또는 “모든 기억을 vector DB 로” 는 둘 다극단이다.

아직 모르는것

- 어떤 deferred job 이 실제 미래 utility 를 만들었는지 측정하는 공통 benchmark 가 없다.
- Google 의 Language Models Need Sleep 은 parametric consolidation 의 직접 증거지만, 공개 실험이 곧 장기 fleet 운영 · 무제한 user 별 weight 성장의 증거는 아니다.
- External-memory sleep 은 이미 product 형태가 보이지만, derived summary 가 원 evidence 를 왜곡하지 않는다는 보장은 없다.
- “얼마나 자주 자야 하나” 는 traffic, staleness, batching, interference 의함수이며 고정 주기로 해결되지 않는다.
- 같은 sleep budget 을 retrieval 개선, synthetic data, adapter training 중 어디에 배분할지에 대한 universal policy 가 없다.

핵심

이 분야의 첫 연구 질문은 “sleep 이 가능한가?”가 아니라 “어떤 reusable state 를 어떤 evidence 와 objective 로, 어느 cadence 에, 얼마나 되돌릴 수있게 바꿀 것인가?” 다.

더 깊이 읽기

- 원 분석: 운영적정의와배포전 ·wake·sleep training map
- Training Background: Wake 상태, Sleep 상태, 손실, 역전파, 옵티마이저
- Claim route: **STC-C005** · synthesis · confidence high · **STC-C006** · synthesis · confidence high · **STC-C015** · synthesis · confidence high

두 개의 독립 축: update timing × memory medium

	Within-query	Post-response	Periodic sleep
Ephemeral / state	KV / recurrent state	session fast state	state compaction
External text · vector · graph	retrieval write	Letta / Mem0	Dreaming / graph rebuild
Parametric weights · adapter	TTT / neural memory	async adapter	replay / distill / refresh

같은 medium도 timing이 다르면 SLA · data horizon · validation이 달라진다

Figure 2: STC-F002. Update timing by memory-medium taxonomy. 원 Study 의저자 작성 합성 figure이며, 사실 주장의 인용은 원 Study 와 figure ledger 를따른다.

문서 경계와 사용법

이 문서는 쉬운 설명본이지 새로운 증거원이 아니다. 수치 · 인용 · claim status 는 원 Study paper 와 claim map 이 우선한다. 학습 개념은 Training Background paper 를 함께 읽고, 논문 재현이나 외부 배포 판단에는 고정 source 와 rights audit 를 사용한다.